01 Einführung

01 Einführung – Softwareentwicklungsprozess

00 Übersicht · Weiter: 01a Programmiersprachen

Vogelperspektive

Softwareentwicklung = Prozess zur Entwicklung eines geeigneten Softwaresystems zur Lösung eines Problems.

`Problem` → (Anforderungen) → `Softwaresystem` → `Lösung`

> **Beispiel aus den Folien**
> Problem: „Wir brauchen ein Softwaresystem, das den Bahnkunden hilft Tickets zu kaufen."
> Lösung: Navigator-App zur Suche nach Verbindungen und zum Kauf von Tickets.

Phasen (Wasserfallmodell)

Das **Wasserfallmodell** ist das älteste und einfachste Vorgehensmodell der Softwareentwicklung.

```
flowchart TD
    A[Anforderungsanalyse] --> B[Entwurf] --> C[Implementierung] --> D[Test] --> E[Auslieferung]
```

> **Begriff**
> **Implementierung** = Programmierung = Realisierung = Umsetzung der Lösung in Code.

Klassen von Programmiersprachen

Es gibt verschiedene Klassen von Programmiersprachen (Programmier-Paradigmen)

- Imperative/prozedurale Programmiersprachen (z.B. Fortran, Cobol, C, Pascal)
  - In der Regel ältere Sprachen (60er bis 80er Jahre), die heute eher selten eingesetzt werden.
  - Aufteilung von komplexen Algorithmen in überschaubare Einheiten (Prozeduren) mit klar definierter Schnittstelle.
  - Die meisten modernen Programmiersprachen unterstützen immer auch prozedurale Aspekte.
- Objektorientierte Programmiersprachen (z.B. Java, C++, C#)
  - Weiterentwicklung der prozeduralen Sprachen in den 90er Jahren.
  - Zentrales Konzept ist die Modularisierung des Codes in Klassen: kapseln Daten (Attributen) und Verhalten (Methoden).
- Funktionale Programmiersprachen (z.B. Lisp, Haskell, F#, Scala)
  - Funktionen stehen im Vordergrund. Daten werden als Parameter übergeben und meist nicht nachträglich verändert.
- Script-Sprachen (z.B. Javascript, Python)
  - Unterstützen in der Regel mehrere der oben genannten Paradigmen.

Der Trend geht heute zu hybriden Sprachen: d.h. Sprachen, die Konzepte verschiedener Paradigmen unterstützen.

7

Syntax, Semantik und Pragmatik von Programmiersprachen

Programmiersprachen sind formale Sprache zur Formulierung von Algorithmen, die von Computern interpretiert und ausgeführt werden können. Sind gekennzeichnet durch:

- Syntax:
  - Gültige Zeichenmenge (Alphabet).
  - Gültige Verknüpfungen der Zeichen des Alphabets zur Bildung von Ausdrücken (Grammatik).
  - Syntax lässt sich gut automatisiert prüfen.
- Semantik:
  - Ordnet Ausdrücken eine konkrete Bedeutung zu.
  - Semantik lässt sich teilweise automatisiert prüfen: statische Prüfung: z.B. korrekte Datentypen durch den Compiler, dynamisches Verhalten zur Laufzeit.
  - Die Semantik wird teilweise erst im Kontext klar (mehrdeutige Ausdrücke).
- Pragmatik:
  - Adäquate Verwendung der Ausdrücke für bestimmte Probleme.
  - Ziel ist die Lesbarkeit und Wartbarkeit des Codes (wie gut kann ein Mensch den Code verstehen und weiterentwickeln).
  - Wird aus der Erfahrung heraus abgeleitet.
  - Lässt sich eingeschränkt automatisiert prüfen, z.B. durch Code Checker.

8

Beispiel Summe 1 bis N

Phase	Inhalt
Anforderungsanalyse	Programm, das die Summe über alle ganzen Zahlen von 1 bis zu einer positiven Obergrenze N berechnet
Entwurf	Lösungsidee, z.B. die **Gaußsche Summenformel** $\sum_{i=1}^{N} i = \frac{N\,(N+1)}{2}$
Implementierung	**allgemeine** Lösung (für unterschiedliche N) in einer Programmiersprache
Test	mit verschiedenen gültigen **und ungültigen** Eingabewerten für N
Auslieferung	Bereitstellung für alle vorgesehenen Nutzer, z.B. Webanwendung in der Cloud, Download, Installation beim Kunden

Code dazu: 01b Java

Take-aways

- Ziel der Softwareentwicklung: digitale Lösungen (Softwaresysteme) für konkrete Probleme der Praxis.
- **Programmieren** = Beschreibung eines Lösungsalgorithmus mit einer Programmiersprache.
- Programmieren ist zentraler, **aber nicht einziger** Bestandteil des Entwicklungsprozesses.